Machine Learning with Scikit-Learn

Oliver Kramer

This practical introduction adds key equations to the main methods and short Python examples. Basic algebra is enough; new symbols are explained as they appear.

1 The basic idea

Machine learning finds patterns in examples to make predictions or explore data.

Task	Question	Example
Classification	Which category?	Which flower species?
Regression	What number?	How much energy is used?
Clustering	Which examples form groups?	Group similar observations.
Dimensionality reduction	Can we use fewer features?	Draw many features in 2D.

Two names you will see everywhere

- `X`: input data. Rows are examples; columns are features.
- `y`: correct answers. Classification uses categories; regression uses numbers.

Learning from `X` and `y` is **supervised learning**. Clustering and dimensionality reduction work with `X` alone: **unsupervised learning**.

Reading the equations

We write n for the number of examples and p for the number of features. x_{ij} is feature j of example i. For one example, we simply write $x = (x_1, \ldots, x_p)$. Its correct answer is y, and $\hat{y}$, pronounced "y-hat", is a prediction.

In the equations, w_j denotes a weight. A subscript identifies an entry; it is not a multiplication. Indices start at 1; Python starts at 0.

Getting ready

The scikit-learn examples are available in this Google Colab notebook.

Install the required packages in your terminal:

```
python -m pip install scikit-learn numpy matplotlib
```

In a notebook, use `%pip install scikit-learn numpy matplotlib` instead.

How to read this script. Run code blocks in order, starting on page 2. Dependencies are stated explicitly. All datasets are built in or generated.

In Python, `import` loads a tool and `print(...)` displays a result.

2 Your first model: K-nearest neighbors

Idea: similar examples often have similar answers. K-nearest neighbors (KNN) finds the closest training examples and lets them vote. With $k = 5$, three votes for class A and two for class B produce a prediction of A.

For feature vectors x and z, the Euclidean distance is

$$\text{dist}(x, z) = \sqrt{\sum_{j=1}^{p} (x_j - z_j)^2}. \tag{1}$$

Subtract matching features, square the differences, add them, and take the square root. For $x = (1, 2)$ and $z = (4, 6)$, the distance is $\sqrt{3^2 + 4^2} = 5$.

KNN uses the k smallest distances and predicts the most frequent class among those neighbors. In our pipeline, distances are measured after scaling.

A complete example

The Iris dataset contains 150 flowers, four measurements per flower, and three species, stored as labels 0, 1, and 2. This code predicts the species from the measurements.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, predictions))
print("Predicted:", predictions[:5])
print("Correct:  ", y_test[:5])
```

1. `train_test_split`: learn from 80%; reserve 20% for testing.
2. The pipeline scales features, then applies KNN.
3. `fit` learns from training data; `predict` returns test labels.
4. `accuracy_score` gives the fraction correct: `0.90` means 90%.

`random_state=42` makes the split repeatable. The number 42 has no special meaning. `stratify=y` keeps class proportions approximately the same in the training and test sets.

Main setting: `n_neighbors`. A small value reacts more to individual examples; a larger value smooths the predictions. KNN is easy to understand, but predicting can become slow with many examples.

3 Support vector machines

Idea: separate classes with a wide margin. We first describe a linear SVM for two classes, labeled -1 and $+1$. Scikit-learn also handles other class labels and problems with more than two classes.

The decision rule

For a feature vector $\mathbf{x} = (x_1, \ldots, x_p)^\top$, define

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = \sum_{j=1}^{p} w_j x_j + b, \qquad \hat{y} = \operatorname{sign}(f(\mathbf{x})). \tag{2}$$

The weights $\mathbf{w}$ determine the boundary's orientation and b shifts it. The superscript $\top$ means transpose; here it lets us write the weighted sum compactly. A positive score predicts $+1$, a negative score predicts -1, and $f(\mathbf{x}) = 0$ is the decision boundary.

Hard margin: perfectly separable data

We choose the scale of $\mathbf{w}$ and b so that the two margin planes (lines in 2D) are $f(\mathbf{x}) = -1$ and $f(\mathbf{x}) = +1$. Their full separation is

$$\text{margin width} = \frac{2}{\|\mathbf{w}\|_2}, \qquad \|\mathbf{w}\|_2 = \sqrt{\sum_{j=1}^{p} w_j^2}. \tag{3}$$

Thus, a smaller weight-vector length gives a wider margin. The hard-margin SVM chooses weights and an intercept by solving

$$\begin{aligned} \min_{\mathbf{w},b} \quad & \frac{1}{2}\|\mathbf{w}\|_2^2 \\ \text{subject to} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \qquad i = 1, \ldots, n. \end{aligned} \tag{4}$$

Read "subject to" as "while satisfying". The constraint places each training example on the correct side, on or beyond its margin line. The supporting examples on those lines are *support vectors*. This formulation requires linearly separable training data.

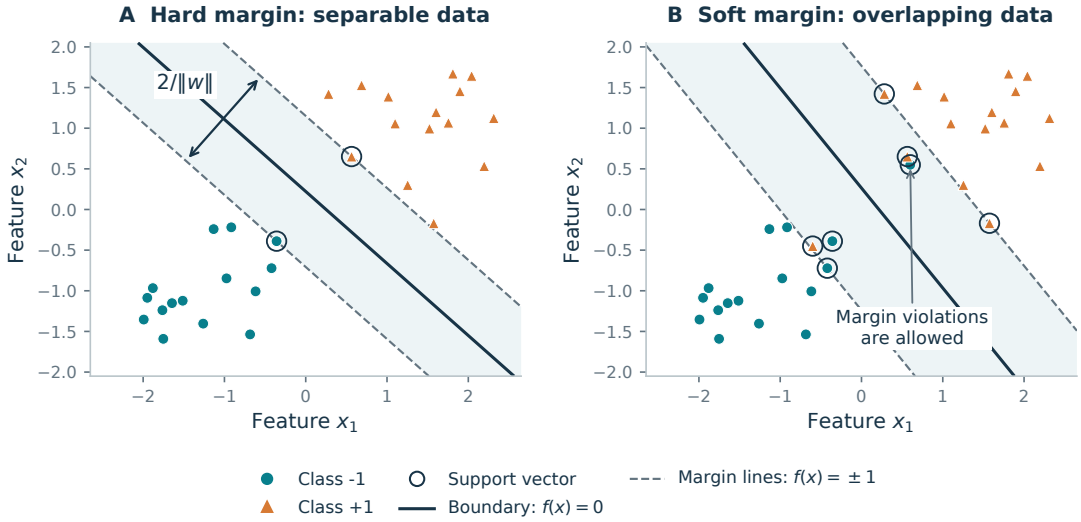


Figure 1: Solid lines show the decision boundary; dashed lines show $f = \pm 1$. Outlined points are support vectors. Left: separable data admit a hard margin. Right: a soft margin allows points inside the band or on the wrong side of the boundary.

Soft margins: allowing violations

For overlapping or noisy data, introduce slack variables $\xi_i \geq 0$. The linear soft-margin SVM minimizes

$$\begin{aligned} \min_{\mathbf{w},b,\boldsymbol{\xi}} \quad & \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{i=1}^{n}\xi_i \\ \text{subject to} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \end{aligned} \tag{5}$$

At the optimum, $\xi_i = 0$ means on or beyond the correct margin; $0 < \xi_i < 1$ means inside it but correctly classified; $\xi_i > 1$ means misclassified. At $\xi_i = 1$, the point lies on the boundary. Larger C penalizes violations more strongly. Choose C by validation; a larger or smaller value is not always better.

Kernels: curved boundaries

A kernel measures similarity through an implicit feature mapping. The RBF kernel used below is

$$K_{\mathrm{RBF}}(\mathbf{x},\mathbf{z}) = \exp\!\left(-\gamma\|\mathbf{x}-\mathbf{z}\|_2^2\right). \tag{6}$$

Nearby points have higher similarity. Larger γ makes this similarity decrease faster with distance, allowing more local detail. A linear separator in the mapped feature space can become a curved boundary in the original coordinates.

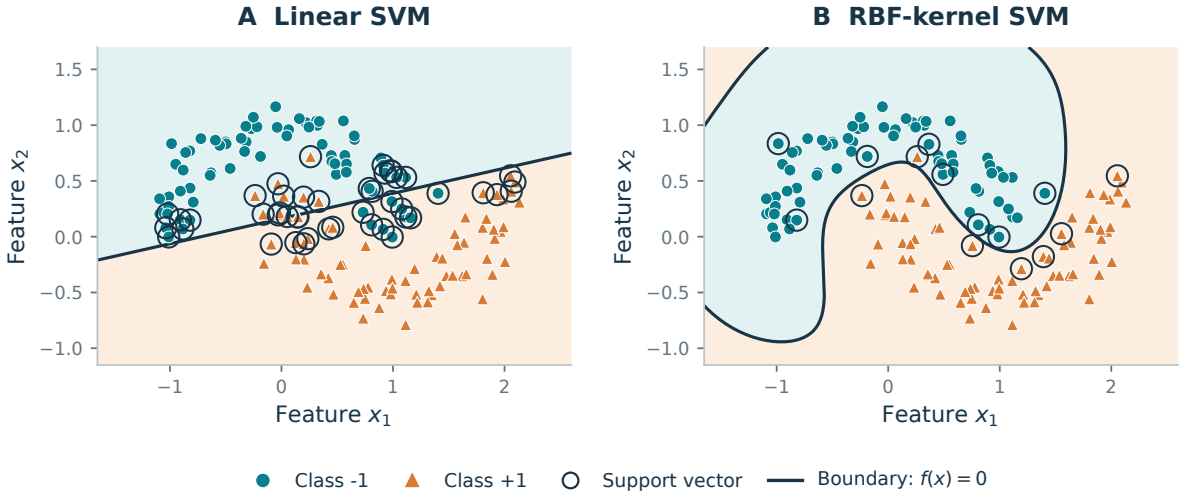


Figure 2: Linear and RBF SVMs fitted to the same two-moons dataset. Shading shows predicted classes, black lines show decision boundaries, and outlines mark support vectors. These are fitted-data illustrations, not estimates of test accuracy.

Reuse the imports and Iris split from page 2:

```
from sklearn.svm import SVC

model = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", C=1.0)
)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
print("SVM accuracy:", accuracy_score(y_test, predictions))
```

4 Random forests

Idea: combine many decision trees. A tree asks a sequence of questions, such as "Is the petal shorter than 2 cm?"

Bootstrapping and bagging

Bootstrapping creates a new training sample by drawing rows *with replacement*: after selecting a row, we can select it again. For each tree, we draw as many rows as the original training set contains. Some rows therefore appear several times, while others are missing. For example, the rows A, B, C, D might give the sample A, C, C, D.

Bagging means *bootstrap aggregating*: train several models on different bootstrap samples, then combine their predictions. This often reduces the variability of predictions compared with one tree.

A random forest also considers a random subset of features at each split. This makes the trees less similar, so their errors can partly cancel when their predictions are combined.

Average the trees' opinions

Let $q_t(c \mid x)$ be the probability that tree t assigns to class c for input x. For T trees, the forest computes

$$\bar{q}(c \mid x) = \frac{1}{T} \sum_{t=1}^{T} q_t(c \mid x).$$

Read $c \mid x$ as "class c, given input x". The forest predicts the class with the largest average probability. Combining different trees often makes predictions more stable than using one tree.

Reuse the Iris split and `accuracy_score` from page 2:

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100, random_state=42
)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
print("Forest accuracy:", accuracy_score(y_test, predictions))
```

This is an average of class probabilities, not necessarily a vote among the trees' final class labels. Feature scaling is usually unnecessary. For regression, a forest instead averages the trees' numerical predictions.

Main settings: `n_estimators` is the number of trees; `max_depth` limits how many successive splits a tree can make. More trees take more time and do not guarantee a better result.

5 Scaling and checking predictions

Why scaling matters

Suppose one feature ranges from 0 to 1 and another from 0 to 10,000. The second can dominate a distance calculation simply because its numbers are larger. For a feature value x, standardization gives

$$x' = \frac{x - \mu}{\sigma}. \tag{7}$$

Here, μ is that feature's training mean and σ its training standard deviation. If $\mu = 10$, $\sigma = 2$, and $x = 14$, then $x' = (14 - 10)/2 = 2$: two standard deviations above the mean.

For nonconstant features, the resulting training values have mean zero and standard deviation one. This does not make the data normally distributed. KNN, SVM, MLP, and Lasso commonly benefit from scaling.

Learn preprocessing from training data only. A pipeline learns the scaler during `fit(X_train, y_train)` and applies that same scaler during prediction. Do not fit a scaler on all data before splitting. Otherwise information from the test set leaks into training.

Accuracy is only one view

If 95 of 100 messages are ordinary mail, always predicting "ordinary" gives 95% accuracy while finding no spam. When class sizes differ, inspect more than accuracy.

Read a confusion matrix

Rows show the true class; columns show the predicted class. Correct predictions sit on the diagonal. Choose a positive class, such as spam. For two classes ordered negative, positive:

	Predicted negative	Predicted positive
Actually negative	True negative (TN)	False positive (FP)
Actually positive	False negative (FN)	True positive (TP)

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN}, & \text{Precision} &= \frac{TP}{TP + FP}, \\ \text{Recall} &= \frac{TP}{TP + FN}, & F_1 &= \frac{2TP}{2TP + FP + FN}. \end{aligned} \tag{8}$$

Precision asks how many positive predictions are correct; recall asks how many actual positives we found. F1 combines both. A zero denominator makes a formula undefined; the code below explicitly reports zero in that case.

Run this after any Iris classifier above. Iris has three classes, so its matrix has three rows and three columns; the report scores each class.

```
from sklearn.metrics import confusion_matrix, classification_report

print(confusion_matrix(y_test, predictions))
print(classification_report(y_test, predictions, zero_division=0))
```

`zero_division=0` reports zero for an undefined precision or recall, for example when a model never predicts a particular class.

6 Predicting numbers: linear regression

Idea: linear regression adds up weighted features to predict a number. With p features, its prediction is

$$\hat{y} = b + \sum_{j=1}^{p} w_j x_j. \tag{9}$$

The intercept b is the starting value; each weight w_j multiplies feature x_j. With one feature, $\hat{y} = b + w_1 x_1$ is a straight line. Training chooses weights and an intercept that minimize mean squared error. This complete example uses the built-in Diabetes dataset.

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.metrics import r2_score

X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, predictions))
print("RMSE:", mean_squared_error(y_test, predictions) ** 0.5)
print("R2:", r2_score(y_test, predictions))
```

Write prediction errors as numbers

For n evaluated examples, let $e_i = y_i - \hat{y}_i$ be the error. The vertical bars in $|e_i|$ mean absolute value: ignore the sign.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |e_i|, \qquad \text{MSE} = \frac{1}{n}\sum_{i=1}^{n} e_i^2, \qquad \text{RMSE} = \sqrt{\text{MSE}}. \tag{10}$$

MAE and RMSE use the target's units; lower is better. RMSE gives large errors more weight. If the correct values are $(3, 5)$ and predictions are $(4, 7)$, the errors are $(-1, -2)$. Then MAE $= (1+2)/2 = 1.5$ and RMSE $= \sqrt{(1+4)/2} \approx 1.58$.

R^2 describes how much of the variation in the target values is accounted for by the model. For nonconstant targets,

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}, \qquad \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i.$$

The denominator measures the **total variation** around the mean; the numerator measures the **unexplained variation**, as squared prediction errors. If these sums are 100 and 20, then $R^2 = 1 - 20/100 = 0.80$: the model explains 80% of the variation. A negative R^2 means the sum of squared prediction errors exceeds the total variation.

7 Lasso: encourage a simpler linear model

Lasso penalizes large weights and can set some weights to exactly zero. This can remove features from the prediction. The penalty is called *regularization*.

Balance fit and simplicity

Lasso uses the same prediction $\hat{y}_i = b + \sum_{j=1}^{p} w_j x_{ij}$ as linear regression, but minimizes a different training objective:

$$\underbrace{\frac{1}{2n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}_{\text{prediction error}} + \underbrace{\alpha \sum_{j=1}^{p} |w_j|}_{\text{penalty on weights}} \quad . \tag{11}$$

Here, n counts training examples. The first term rewards accurate predictions. The second rewards small weights, with α controlling its strength. This $1/(2n)$ convention matches scikit-learn's Lasso; it is half the training MSE. The intercept b is not penalized.

Because weights depend on feature units, scale the inputs before applying the penalty. Reuse the Diabetes split and metric import from the preceding section:

```
from sklearn.linear_model import Lasso
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(StandardScaler(), Lasso(alpha=1.0))
model.fit(X_train, y_train)
predictions = model.predict(X_test)
print("Lasso MAE:", mean_absolute_error(y_test, predictions))
print("Weights:", model.named_steps["lasso"].coef_)
```

Main setting: `alpha`. Larger values encourage smaller weights and often more zero weights. Too much regularization can make the model too simple. Choose the value with cross-validation.

> If a fitted weight is exactly zero, its feature contributes nothing to this model's prediction. That is how Lasso can select features. A zero weight does not establish that a feature is irrelevant in every setting. Linear regression and Lasso can both miss curved relationships.

8 Fitting curves: polynomial regression

Idea: give linear regression extra features such as x^2 and x^3, so it can fit a curve. Degree 1 gives a line, degree 2 allows a quadratic curve, and degree 3 allows a cubic curve. For degree 2,

$$\hat{y} = b + w_1 x + w_2 x^2. \tag{12}$$

The curve can bend as x changes, but the prediction is still a weighted sum of the features x and x^2. This is why we can use linear regression after creating polynomial features.

This complete example generates a curved dataset and fits a quadratic. The target follows $y = x^2 + \text{noise}$.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

rng = np.random.default_rng(42)
X = np.linspace(-3, 3, 100).reshape(-1, 1)
y = X[:, 0] ** 2 + rng.normal(0, 1, 100)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = make_pipeline(
    PolynomialFeatures(degree=2, include_bias=False),
    LinearRegression()
)
model.fit(X_train, y_train)
print("MAE:", mean_absolute_error(y_test, model.predict(X_test)))

plt.scatter(X_train[:, 0], y_train, label="Training examples")
plt.scatter(X_test[:, 0], y_test, label="Test examples")
plt.plot(X[:, 0], model.predict(X), color="black", label="Model")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()
```

`reshape(-1, 1)` makes a table with one column, as scikit-learn expects. `X[:, 0]` selects that column as a one-dimensional array. `include_bias=False` avoids adding a constant feature because `LinearRegression` already fits an intercept.

Underfitting and overfitting

- **Underfitting:** the model is too simple to capture the pattern. A straight line cannot capture this U-shaped relationship.
- **Overfitting:** the model fits training noise as well as the pattern. Its training error looks good, but new-data error is worse.

Main setting: `degree`. More flexibility is not automatically better. Select the degree using training-only validation, then check the chosen model on the final test set. A high degree alone does not prove overfitting.

9 A small neural network: multilayer perceptron

Idea: pass features through layers of neurons. Hidden layers apply nonlinear transformations, allowing complex patterns. Training adjusts weights to reduce prediction error.

One hidden neuron

A hidden neuron with a ReLU activation computes

$$h = \max\left(0,\ b + \sum_{j=1}^{p} w_j x_j\right). \tag{13}$$

The neuron forms a weighted sum plus an offset b, then replaces negative results by zero. This ReLU activation lets the network learn nonlinear patterns.

A neural network groups these neurons into layers. The layers are arranged one after another, with each layer passing its outputs to the next.

We return to Iris. This block loads and splits it again, so it can also run after the regression examples.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = make_pipeline(
    StandardScaler(),
    MLPClassifier(
        hidden_layer_sizes=(20,),
        max_iter=2000,
        random_state=42
    )
)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
print("MLP accuracy:", accuracy_score(y_test, predictions))
```

Layer sizes and training time

- `hidden_layer_sizes=(20,)`: one hidden layer with 20 neurons. The comma makes this a one-element Python tuple.
- `hidden_layer_sizes=(50, 20)`: two hidden layers, with 50 and 20 neurons. A larger network can learn more complex patterns but needs more computation and may overfit.
- `max_iter` limits training iterations. If training does not converge, check scaling and consider increasing it; more iterations do not guarantee better predictions.

> Scale MLP inputs and compare models using cross-validation. For early stopping, use validation performance; keep the final test set untouched.

`MLPClassifier` predicts categories. The related `MLPRegressor` predicts numbers.

10 Finding groups: DBSCAN

Idea: connect examples in dense neighborhoods and leave isolated examples unassigned. DBSCAN does not need the number of clusters in advance. It can find curved groups, such as the two moons below.

Two settings define a dense neighborhood

- `eps`: the distance used to decide which examples are nearby.
- `min_samples`: how many examples a neighborhood needs for its center to be a *core point*. This count includes the point itself.

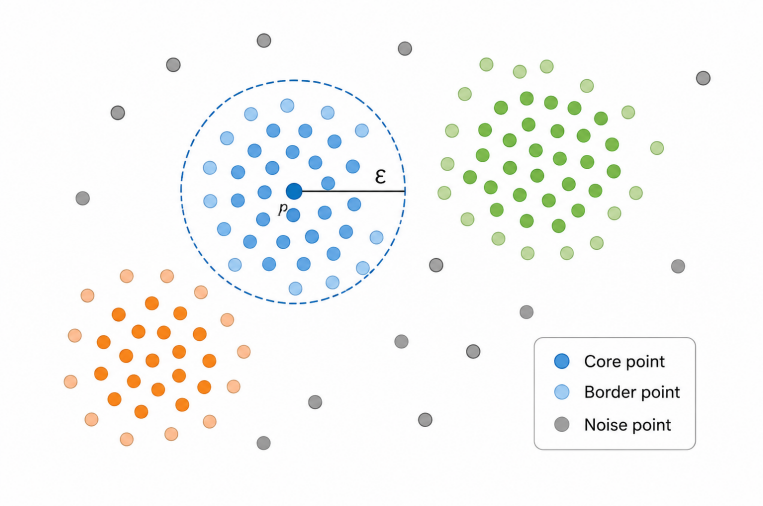


Figure 3: Illustration of the DBSCAN algorithm. A core point has at least `min_samples` neighbors within the radius ε. Neighboring core points form clusters, border points are assigned to nearby clusters, and isolated points are classified as noise.

In symbols, let $N_\varepsilon(x_i)$ contain all examples whose distance from x_i is at most ε, the value of `eps`. Then

$$x_i \text{ is a core point if } |N_\varepsilon(x_i)| \geq \texttt{min_samples}. \tag{14}$$

Here the vertical bars count the examples in the neighborhood, including x_i itself. They do not mean absolute value when used around a set.

Nearby core points connect to form clusters. A *border point* is close to a core point but is not itself dense enough to be a core point. Other points are marked as *noise*.

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN

X, _ = make_moons(n_samples=300, noise=0.08, random_state=42)
labels = DBSCAN(eps=0.25, min_samples=5).fit_predict(X)

n_clusters = len(set(labels) - {-1})
print("Clusters:", n_clusters)
print("Noise points:", (labels == -1).sum())

plt.scatter(X[:, 0], X[:, 1], c=labels, cmap="viridis")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("DBSCAN: each color is a cluster or noise label")
plt.show()
```

`fit_predict(X)` learns the grouping and returns a label for each input row. Labels such as 0 and 1 are arbitrary cluster names. **Label -1 means noise**, so it is excluded from the cluster count. The underscore in `X, _` means we do not use the generator's true labels.

What happens when the settings change?

- A smaller `eps` usually gives smaller groups and more noise.
- A larger `eps` can connect groups that should stay separate.
- A larger `min_samples` demands a denser neighborhood.

DBSCAN uses distances, so feature units matter. The generated moon coordinates already have similar scales. For other data, consider scaling first and then choose `eps` in the scaled space. DBSCAN can struggle when different clusters have very different densities.

This is an exploratory grouping, not a classifier trained on known answers. Scikit-learn's DBSCAN does not provide a general `predict` method for assigning new examples after fitting.

11 Seeing data in two dimensions: PCA

Idea: replace many features with a few new coordinates. Imagine a cloud of points: PCA finds the directions in which the cloud spreads out most. Keeping two directions lets us draw the data in 2D.

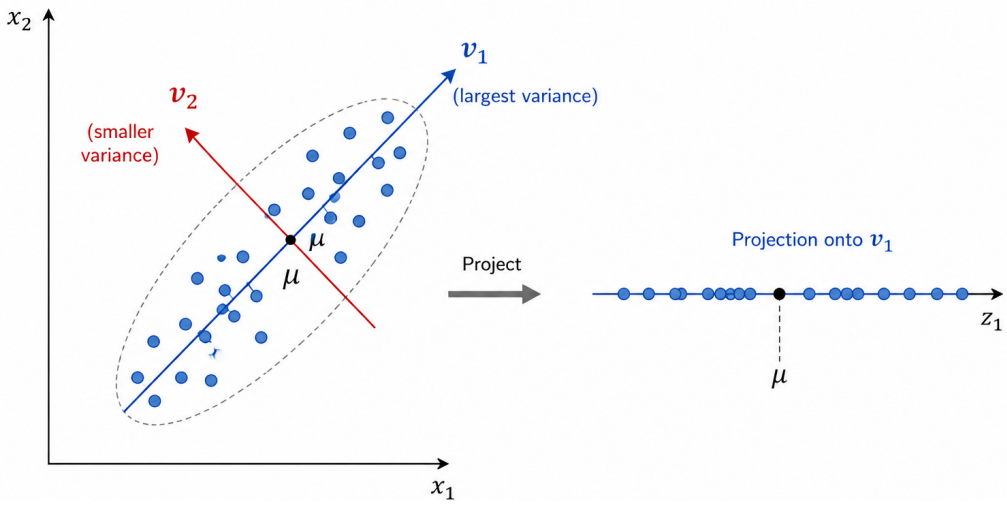


Figure 4: Illustration of Principal Component Analysis (PCA). PCA finds new orthogonal axes that capture the directions of largest variance in the data. The data are then projected onto the first principal components to obtain a lower-dimensional representation.

1. Center the data

Let $x = (x_1, \ldots, x_p)$ describe one example and let μ_j be the mean of feature j. Subtract the means:

$$x_j^{\text{centered}} = x_j - \mu_j.$$

This moves the center of the cloud to zero. If features have different scales, standardize them first, as in the example below.

2. Find new directions

The first principal direction follows the greatest spread of the data. The second is perpendicular to the first and follows the greatest remaining spread. PCA learns these directions from the features alone, without using the class labels.

3. Calculate the new coordinates

For a unit-length direction $v = (v_1, \ldots, v_p)$, the new coordinate is

$$z = \sum_{j=1}^{p} v_j (x_j - \mu_j).$$

This is a weighted sum of the centered features. Using two directions gives two coordinates per example, which become the horizontal and vertical positions in the plot.

PCA in scikit-learn

This example turns the four Iris measurements into two coordinates. Scikit-learn finds the directions and computes the new coordinates for us.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X, y = load_iris(return_X_y=True)
X_scaled = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
Z = pca.fit_transform(X_scaled)

plt.scatter(Z[:, 0], Z[:, 1], c=y, cmap="viridis")
plt.xlabel("Principal component 1")
plt.ylabel("Principal component 2")
plt.title("Iris: PCA")
plt.show()
```

Main setting: `n_components`. Here it is 2, so `Z` has two columns. Reducing the number of coordinates can lose information.

PCA centers its input but does not automatically standardize feature scales. This plot explores all rows, and `y` only chooses colors. For predictive evaluation, put scaling and PCA inside a pipeline fitted on training data only.

12 MDS: approximately preserve distances

Idea: place examples on a map so the distances between them resemble their original distances.

Measure how much the map distorts distances

Let d_{ij} be the distance between examples i and j before mapping, and $\hat{d}_{ij}$ their distance on the 2D map. Metric MDS tries to make the following quantity small:

$$\text{Stress} = \sum_{i<j} (d_{ij} - \hat{d}_{ij})^2. \tag{15}$$

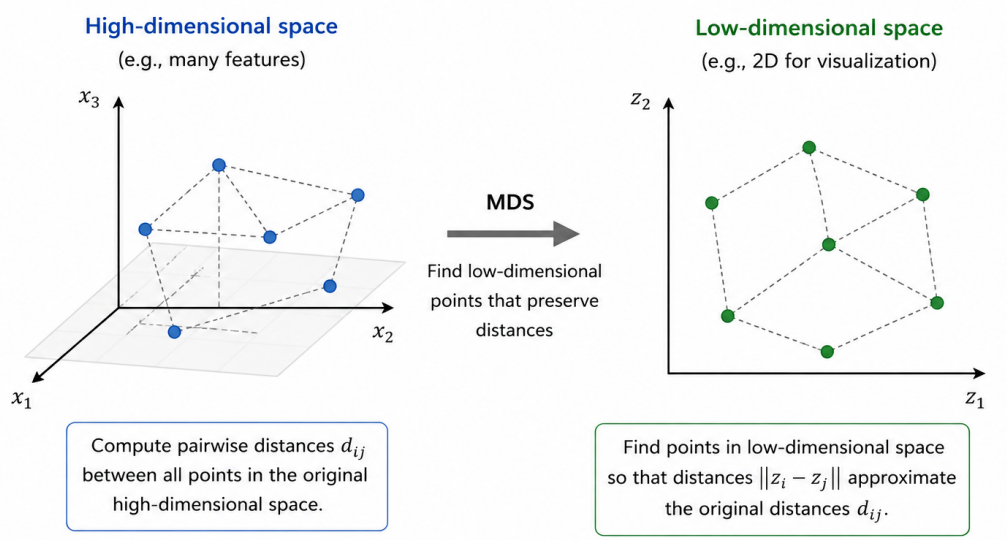


Figure 5: Illustration of Multidimensional Scaling (MDS). MDS first computes the pairwise distances between all samples in the original high-dimensional space and then finds a low-dimensional embedding whose distances approximate the original distances as closely as possible. The quality of the embedding is measured by the *stress* function.

The sum covers every distinct pair once. Each squared difference measures how much that pair's distance changed. A perfect distance-preserving map would have zero stress, but this may be impossible in only two dimensions. This formula is raw stress; normalized variants also exist.

Reuse `X_scaled`, `y`, and the plotting import from the PCA example:

```
from sklearn.manifold import MDS

mds = MDS(n_components=2, n_init=4, random_state=42)
Z = mds.fit_transform(X_scaled)
plt.scatter(Z[:, 0], Z[:, 1], c=y, cmap="viridis")
plt.xlabel("MDS dimension 1")
plt.ylabel("MDS dimension 2")
plt.title("Iris: MDS")
plt.show()
```

MDS is often slower than PCA. Its *stress* measures distance distortion; lower stress means a closer match for the same data and settings. MDS focuses on distances between points rather than finding principal directions.

	PCA	Metric MDS
Main idea	Retain variance in a few linear directions.	Match pairwise distances.
What is learned?	Axes for projecting examples.	Positions of the fitted examples.

These plots explore the whole dataset, so scaling all rows here is intentional. The labels only choose plot colors. For predictive evaluation, put scaling and PCA inside a pipeline fitted on training data only. Visual separation does not establish test accuracy.

13 PCA before classification

Idea: use reduced coordinates as the inputs to a classifier:

$$\text{original features} \longrightarrow \text{reduced coordinates} \longrightarrow \text{predicted class.}$$

PCA and MDS learn coordinates without class labels; the classifier learns from the coordinates and labels. Fewer features may simplify classification, but can also remove useful information. Check validation accuracy.

PCA inside a classification pipeline

The Wine dataset has 13 features. This example compares KNN with two PCA components, five components, and all 13 scaled features without PCA. Cross-validation is explained on page 16.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = make_pipeline(
    StandardScaler(), PCA(), KNeighborsClassifier(n_neighbors=5)
)
settings = {"pca": [
    "passthrough", PCA(n_components=2), PCA(n_components=5)
]}
search = GridSearchCV(model, settings, cv=3, scoring="accuracy")
search.fit(X_train, y_train)

print("Best preprocessing:", search.best_params_)
print("Validation accuracy:", search.best_score_)
print("Test accuracy:", search.score(X_test, y_test))
```

`"passthrough"` skips PCA. Each fold learns its own scaler and, when used, PCA directions from its training rows. The best pipeline is refitted on the full training set; new samples then use that same scaler and projection. For classification, the number of components need not be two.

Why MDS needs an extra step

MDS coordinates can also be classifier inputs: KNN would find neighbors in the MDS map. However, scikit-learn's `MDS` has `fit_transform` but no `transform` for new samples, so it cannot directly replace PCA in this pipeline. A predictive MDS workflow needs an additional method that places new samples in the fixed training map.

> Split before fitting preprocessing. Fitting MDS on training and test rows together lets test data influence the map. Fitting separate maps gives incompatible coordinates for the classifier. PCA provides a reusable projection for new rows.

14 Choosing a model setting

Settings chosen before training, such as KNN's number of neighbors, are called *hyperparameters*. To compare settings without using the test answers, use **cross-validation on the training set**.

In 3-fold cross-validation, the training set is divided into three parts. Each part becomes the validation set once while the other two are used for training. We average the three validation scores.

For K folds with scores $s_1, \ldots, s_K$,

$$\bar{s} = \frac{1}{K} \sum_{r=1}^{K} s_r. \tag{16}$$

Here, K is the number of folds, distinct from KNN's k neighbors.

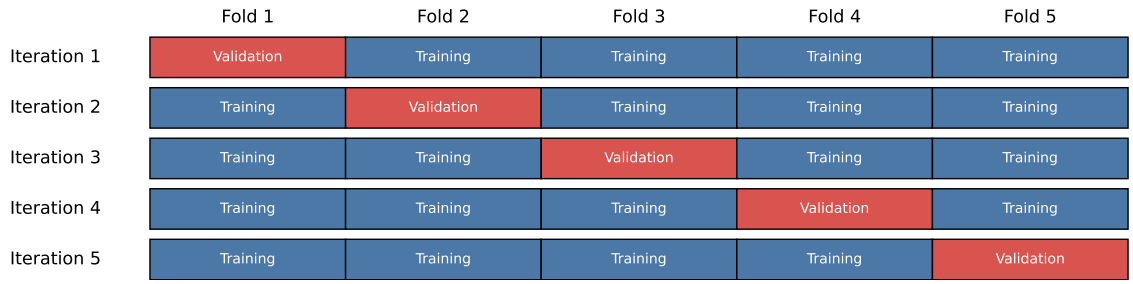


Figure 6: K-fold cross-validation. The training set is divided into K folds. Each fold is used once for validation while the remaining folds are used for training. The average validation score is used to select the best hyperparameters.

Let a small grid search choose k

`GridSearchCV` tries each listed setting using cross-validation. This complete example searches only three values, keeping the process easy to follow. The scaler is inside the pipeline, so it is also fitted separately within every training fold.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
model = make_pipeline(StandardScaler(), KNeighborsClassifier())
settings = {"kneighborsclassifier__n_neighbors": [3, 5, 9]}

search = GridSearchCV(model, settings, cv=3, scoring="accuracy")
search.fit(X_train, y_train)

print("Best setting:", search.best_params_)
print("Mean validation accuracy:", search.best_score_)
print("Final test accuracy:", search.score(X_test, y_test))
```

The long setting name means `pipeline_step_name__parameter_name`: there are two underscores between the step and the parameter. `make_pipeline` creates step names from the lowercase class names.

After choosing the best setting, `GridSearchCV` automatically trains that pipeline again on the full training set. The final line then checks it on the held-out test set.

Training learns; validation chooses; testing checks. Do not keep changing the model to improve the same test score. That turns the test set into another validation set and makes the result too optimistic. A tiny score difference on Iris is not proof that one method is generally better.

The random splits here are suitable for these independent-example demos. For time-ordered data or repeated observations from the same person, use a split that respects time or keeps related observations together.

15 Exercises and a quick reference

Five short exercises

1. **Scale and tune KNN.** Load the Wine dataset. Compare KNN with $k = 5$ with and without scaling using cross-validation. Try `[1, 3, 5, 9, 15]` in a scaling–KNN grid search. Report the best k and test accuracy. Explain why scaling matters.
2. **Compare classifiers.** Load the Breast Cancer dataset. Compare a scaled SVM and a Random Forest using the same training folds. Evaluate the chosen model on the test set using a confusion matrix, accuracy, precision, recall, and F1. Treat malignant cases as positive. Which metric measures how many malignant cases are detected?
3. **Predict a number.** Load the Diabetes dataset. Compare linear regression with a scaling–Lasso pipeline, choosing α by cross-validation. Report test MAE, RMSE, and R^2. Count the zero Lasso coefficients and explain how the penalty simplifies the model.
4. **Find groups.** Create 300 Two-Moons samples (noise 0.08). Use DBSCAN with `min_samples=5` and `eps` values 0.1, 0.25, and 0.5. Plot the results and count clusters and noise points. Explain how changing `eps` affects the grouping.
5. **Visualize data.** Select 500 random samples from Digits and standardize the features. Compare two-dimensional PCA and MDS plots, colored by digit. Compare computation times. Explain what each method aims to preserve.

Which tool does what?

Method	Remember the idea	First setting
KNN	Ask nearby training examples.	`n_neighbors`
SVM	Separate classes with a margin.	`C`
Random Forest	Combine many decision trees.	`n_estimators`
Linear regression	Add weighted features.	Usually defaults
Lasso	Shrink linear-model weights.	`alpha`
Polynomial regression	Add powers to fit curves.	`degree`
MLP	Learn weights across layers.	`hidden_layer_sizes`
DBSCAN	Connect dense neighborhoods.	`eps`
PCA	Keep directions of large variance.	`n_components`
MDS	Preserve distances on a map.	`n_components`

The main scikit-learn verbs

`fit`: learn from data. `predict`: produce answers.
`transform`: change the representation of data.
`fit_transform`: learn the transformation and apply it.
`fit_predict`: learn a grouping and return labels for the fitted data.

A Optional course challenge

The original script includes the Machine Learning Challenge 2026. This appendix keeps its submission requirements in one place. The challenge files are separate course materials; they are not needed for the examples in this beginner script.

What the files contain

- `train.csv`: feature columns, an `id`, and a target column.
- `test.csv`: feature columns and `id`, but no target.
- `submission_template.csv`: the required output structure.

Treat `id` as the row identifier. Keep it for the submission and use the feature columns as model inputs.

The five tasks

The titles are shuffled codenames, not descriptions of the data-generating mechanisms. The original course notes say the mechanisms will be revealed in the final tutorial.

Challenge folder	Task	Metric
`challenge_01_the_final_boss`	Classification	Accuracy
`challenge_02_the_scaling_trap`	Regression	RMSE
`challenge_03_three_worlds`	Classification	Accuracy
`challenge_04_needle_in_a_haystack`	Classification	F1
`challenge_05_hidden_xor`	Classification	F1

A simple workflow

1. Use only the supplied data. Split labeled training data for validation.
2. Build a reproducible workflow that includes preprocessing and fitting.
3. Compare models using the challenge's metric. Different challenges may use different models. Record the validation strategy and estimated score.
4. Once the choice is fixed, refit on all labeled training rows and predict the supplied test rows. The hidden test labels are used only for the final evaluation.

Submit one CSV per challenge with exactly two columns:

```
id,prediction
```

For classification, submit class labels 0 or 1. For regression, submit finite numerical predictions. Include every test `id` exactly once, in the supplied order. Accuracy and F1 are better when higher; RMSE is better when lower. F1 is computed for class 1.

Further reading

This edition is adapted from the supplied lecture manuscript. For details, see the official scikit-learn introduction, preprocessing and evaluation pitfalls, the Lasso objective, and the MDS reference.